

UNIT STUDY NOTES

Jupyter Demo: Working Effectively in Notebooks

Jupyter is the working surface for this entire course. This unit covers the execution model, keyboard workflow, cell state, output inspection and the hygiene rules that keep a notebook reproducible six months later.

 1 source notebook 18 topic sections extracted 6 study sections

Unit Overview

Command Mode (press **Esc** to enable)

- **Enter**: enter edit mode
- **Shift** + **Enter**: run the cell, select below
- **Ctrl** + **Enter**: run the cell
- **Alt** + **Enter**: run the cell, insert below
- **Y**: change cell to code
- **M**: change cell to markdown
- **A**: insert cell above
- **B**: insert cell below
- **D**, **D** (press the key twice): delete selected cell
- **Z**: undo cell deletion
- **Shift** + **M**: merge selected cells, or current cell with cell below if only one cell is selected
- **Ctrl** + **S**: save and checkpoint
- **L**: toggle line numbers
- **Shift** + **L**: toggle line numbers in all cells (when in command mode)
- **Up** or **K**: select cell above
- **Down** or **J**: select cell below

Edit Mode (press **Enter** to enable)

- **Esc**: enter command mode
- **Shift** + **Enter**: run the cell, select below
- **Ctrl** + **Enter**: run the cell
- **Alt** + **Enter**: run the cell, insert below
- **Ctrl** + **Shift** + **-**: split the cell at cursor
- **Ctrl** + **Z**: undo last text edit

- `Ctrl` + `Shift` + `Z` or `Ctrl` + `Y`: redo last text edit
- `Ctrl` + `Home`: go to cell start
- `Ctrl` + `End`: go to cell end
- `Ctrl` + `Left`: go one word left
- `Ctrl` + `Right`: go one word right
- `Ctrl` + `Backspace`: delete word before
- `Ctrl` + `Delete`: delete word after
- `Tab`: code completion or indent
- `Shift` + `Tab`: tooltip
- `Ctrl` + `]`: indent
- `Ctrl` + `[`: dedent
- `Ctrl` + `A`: select all
- `Ctrl` + `Shift` + `K`: comment out the selected lines

Prerequisites

- A Python 3 installation

What you will be able to do

- ✓ Understand the kernel/client split and why cell execution order matters
- ✓ Navigate and edit cells with confidence in command and edit mode
- ✓ Inspect and suppress output deliberately instead of by accident
- ✓ Keep notebooks readable: named cells, markdown structure, reproducible runs

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

Keyboard Shortcuts

Using jupyter as command line

Display all the rows and columns

PYTHON

```
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

df
```

PYTHON

```
pd.set_option('display.max_rows', None)
```

Supress Warning

PYTHON

```
import pandas as pd

# Creating a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Trying to modify a slice of the DataFrame directly
subset = df[df['A'] > 1]
subset['B'] = subset['B'] + 1
```

PYTHON

```
import warnings
warnings.filterwarnings('ignore')
```

Print all variable

What/Why

Variables reference objects in memory. Data types determine operations allowed and behavior.

Semantics

Python is dynamically typed. Assignment binds names to objects; mutability matters.

Common mistakes

Confusing assignment vs comparison, mutability side effects.

PYTHON

```
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
```

PYTHON

```
df.describe()
df.info()
```

Print output of any other cell

PYTHON

```
Out[7]
```

Close long output

PYTHON

```
Out
```

Remove extra output

PYTHON

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [4, 5, 6])
```

PYTHON

```
plt.plot([1, 2, 3], [4, 5, 6]);
```

Magic Commands

% is used for **line magics**. Line magic commands apply only to the rest of the line on which they are used.

%% is used for **cell magics**. Cell magic commands apply to the entire cell in which they are the first line. This means that everything below the %% command (up to the end of the cell) is considered part of the command. Cell magics allow you to write more complex or multi-line commands, affecting or transforming the entire cell's content.

%time

PYTHON

```
# Example of using %time for a single line of code
%time sum(range(1000000))
```

%run

%% writefile

%%html

%%javascript

%%latex

Jupyter Widgets

PYTHON

```
import ipywidgets as widgets
from IPython.display import display

slider = widgets.IntSlider(
    value=7,
    min=0,
    max=10,
    step=1,
    description='Test Slider:',
    disabled=False,
    continuous_update=False,
    orientation='horizontal',
    readout=True,
    readout_format='d'
)
display(slider)
```

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from ipywidgets import interactive

def plot_sine_wave(frequency=1.0):
    # Generate x values
    x = np.linspace(0, 2 * np.pi, 100)
    # Calculate y values based on the current frequency
    y = np.sin(x * frequency)

    # Plotting the sine wave
    plt.figure(figsize=(10,4))
    plt.plot(x, y)
    plt.ylim(-1.1, 1.1)
    plt.title('Sine Wave')
    plt.show()

# Create an interactive widget for the sine wave frequency
interactive_plot = interactive(plot_sine_wave, frequency=(1, 10, 0.1))
display(interactive_plot)
```

Jupyter Extensions

PYTHON

```
# Hinterland
import pandas as pd
import matplotlib.pyplot as plt
```

ChatGPT Extension

PYTHON

```
# Explain

def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n-i-1):
            # Traverse the array from 0 to n-i-1. Swap if the element found is greater
            # than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = bubble_sort(arr)
print("Sorted array is:", sorted_arr)
```

PYTHON

```
# debug

def buggy_bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        # Intentional bug: Should be 'range(n-i-1)'
        for j in range(n):
            # Intentional bug: Missing condition to avoid index out of range error
            if arr[j] > arr[j+1]:
                # Intentional bug: Incorrect swapping logic
                temp = arr[j]
                arr[j] = arr[j+1]
                arr[j+1] = temp
    return arr

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = buggy_bubble_sort(arr)
print("Sorted array is:", sorted_arr)
```

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- kernel state and re-execution order
- command mode versus edit mode
- output and display control
- notebook hygiene and reviewability

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

DISPLAY ALL THE ROWS AND COLUMNS

```
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

df
```

PYTHON

DISPLAY ALL THE ROWS AND COLUMNS

```
df = pd.read_csv('https://raw.githubusercontent.com/prasertcbs/basic-dataset/master/MNIST.csv')

df
```

PYTHON

SUPPRESS WARNING

```
import pandas as pd

# Creating a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Trying to modify a slice of the DataFrame directly
subset = df[df['A'] > 1]
subset['B'] = subset['B'] + 1
```

PYTHON

SUPRESS WARNING

```
import warnings
warnings.filterwarnings('ignore')
```

PYTHON

PRINT ALL VARIABLE

```
df.describe()
df.info()
```

PYTHON

PRINT ALL VARIABLE

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

PYTHON

PRINT ALL VARIABLE

```
df.describe()
df.info()
a=2
a
```

PYTHON

REMOVE EXTRA OUTPUT

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [4, 5, 6])
```

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

☐ Using jupyter as command line

☐ Display all the rows and columns

☐ Supress Warning

☐ Print all variable

☐ Print output of any other cell☐ Close long output☐ Remove extra output☐ Magic Commands☐ %time☐ %run☐ %% writefile☐ %%html☐ %%javascript☐ %%latex☐ Jupyter Widgets☐ Jupyter Extensions☐ ChatGPT Extension